Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI5CC · CLOUD COMPUTING · WEEK 4

Cloud Scaling and Load Balancing

Elasticity, high availability, load balancing, Auto Scaling and containers on Kubernetes.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Recap

LAST SESSION

Virtualization

Virtual machines, hypervisors and containers.

LAST SESSION

Amazon EC2

Instances, instance types, pricing, AMIs and snapshots.

LAST SESSION

Lab 1

We launched one EC2 instance and connected to it with SSH.

One instance is one point of failure, and one fixed size. Today we fix both.

Learning objectives

1. Compare horizontal and vertical scaling.
2. Explain elasticity and high availability.
3. Describe load balancer types and routing.
4. Explain Auto Scaling groups, policies and metrics.
5. Explain how Kubernetes, ECS and EKS scale containers.
6. Choose a scaling tool for a given workload.
7. Set up auto scaling behind a load balancer (Lab 2).

THIS WEEK

Topic 4

Cloud scaling and load balancing. Assessed by class participation, a quiz and a lab report (CLO1, CLO2).

DISCUSSION

What happens when everyone arrives at once?

A ticket sale opens, exam results are published, a video goes viral. One server, thousands of users.

Week 4

PART 1

Scaling and elasticity

Adjusting resources to meet the application’s needs.

Week 4

Scalability, elasticity and agility

CAN WE GROW?

Scalability

The ability of a system to handle more load by adding resources. It answers: can we grow?

CAN WE GROW AND SHRINK?

Elasticity

The ability to add and remove resources automatically as demand rises and falls. It answers: can we grow and shrink, without anyone doing it by hand?

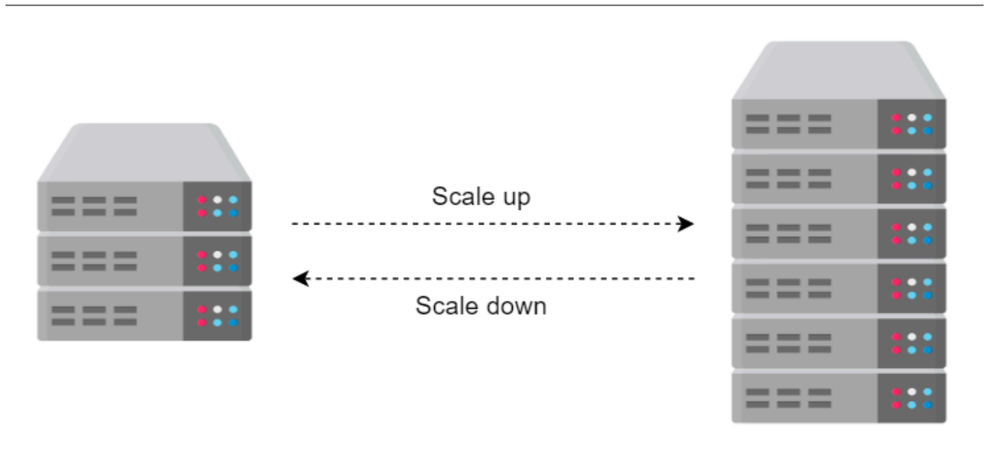
HOW FAST CAN WE START?

Agility

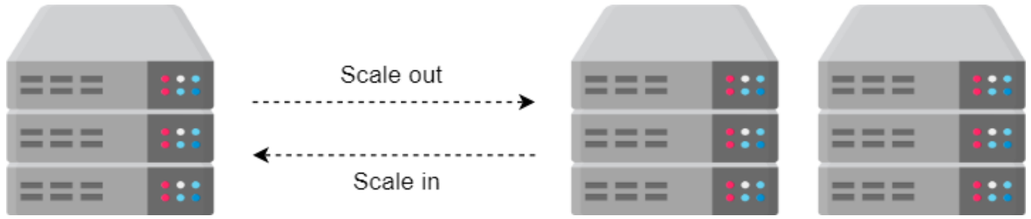
New resources are a few clicks away, so they arrive in minutes, not weeks. Often mixed up with the other two, but it says nothing about handling load.

Rapid elasticity is one of the five essential characteristics of cloud computing from Week 1.

Vertical scaling vs horizontal scaling



Vertical: a more powerful machine



Horizontal: more machines of the same size

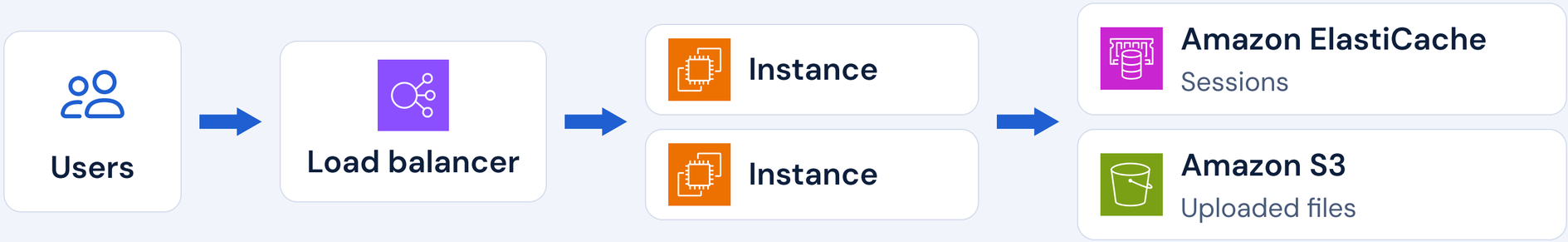
Scale up or down changes the size of one machine. Scale out or in changes the number of machines.

Vertical vs horizontal scaling

	Vertical (scale up)	Horizontal (scale out)
How	A bigger instance type	More instances of the same type
Limit	The largest machine available	Practically none
Downtime	Usually a stop and restart	None: instances join and leave
If one machine fails	Everything is down	The others keep serving
What it needs	Nothing from the application	A load balancer and a stateless application

Horizontal scaling needs stateless servers

Any instance must be able to answer *any request*.



The problem

If a user's session or uploaded file lives on one instance, the next request may land on another instance that has never seen it.

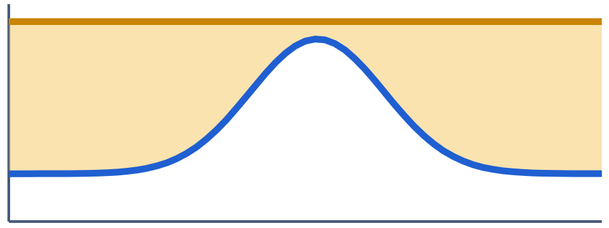
The fix

Keep state outside the instances: sessions in a database or cache, files in shared storage such as Amazon S3.

The result

Instances become interchangeable. They can be added, removed or replaced at any time.

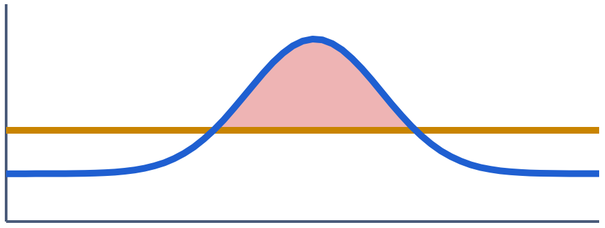
Why elasticity saves money



OVERPROVISIONED

Fixed capacity for the peak

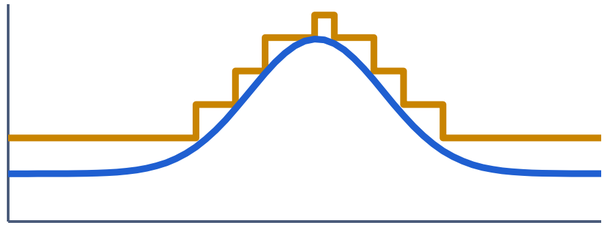
We pay all day and all night for servers we only need at the busiest hour.



UNDERPROVISIONED

Fixed capacity for the average

Cheap, until the peak arrives and users see errors and slow pages.



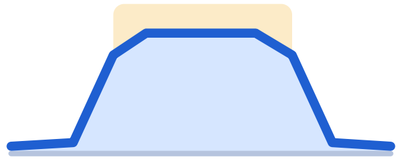
JUST RIGHT

Elastic capacity

Capacity follows demand. We pay for what the load needs, hour by hour.

Demand over one day · Capacity we pay for · Wasted capacity · Users turned away

Four traffic patterns, four scaling choices



DAILY

Predictable, every day

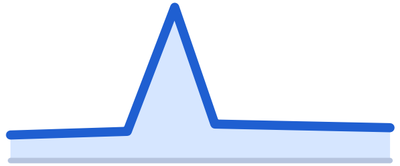
A campus portal busy from 8:00 to 17:00. Use scheduled scaling.



EVENT

Predictable, one time

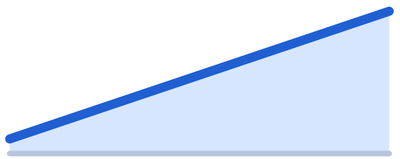
Registration opens on a known date. Raise the capacity ahead of time.



SURPRISE

Unpredictable

A post goes viral without warning. Use target tracking, which reacts to a metric.



GROWTH

Steady growth

A new app gains users month by month. Review the maximum regularly.

Real systems mix them: a schedule for the peak we know about, and target tracking for the one we do not. The curves are drawings, not measurements.

It is not only instances: what scales on AWS



Amazon EC2

WHAT SCALES

The number of instances

HOW

EC2 Auto Scaling groups



Amazon ECS and EKS

WHAT SCALES

Tasks and pods

HOW

Service auto scaling, Kubernetes autoscalers



AWS Lambda

WHAT SCALES

Concurrent executions

HOW

Automatic, one per request



Amazon DynamoDB

WHAT SCALES

Read and write capacity

HOW

On-demand mode, or auto scaling



Amazon Aurora

WHAT SCALES

Read capacity

HOW

Auto scaling of read replicas



Amazon S3

WHAT SCALES

Storage and requests

HOW

Automatic, nothing to set

One idea, many services. We start with EC2, then see containers in Part 5 and Lambda again in Week 9.

PART 2

High availability

Keeping the application reachable when something fails, because something always does.

Week 4

High availability concepts



Single point of failure

Any one component whose failure takes the whole application down. High availability means removing them.



Redundancy

More than one of everything that matters, so a spare is already running when one fails.



Health checks

Regular tests that detect a failed component, so traffic stops going to it.



Failover

Moving the work to a healthy component automatically, without waiting for a person.

What the nines mean

AVAILABILITY
99%
DOWNTIME PER YEAR
3.65 days
PER 30-DAY MONTH
7.2 hours

÷ 10 →

AVAILABILITY
99.9%
DOWNTIME PER YEAR
8.76 hours
PER 30-DAY MONTH
43.2 minutes

÷ 10 →

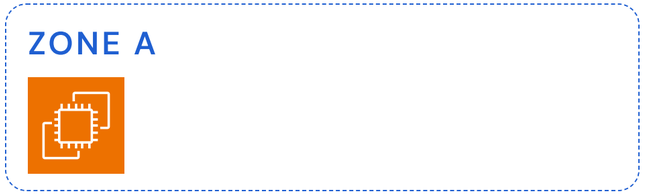
AVAILABILITY
99.99%
DOWNTIME PER YEAR
52.6 minutes
PER 30-DAY MONTH
4.3 minutes

÷ 10 →

AVAILABILITY
99.999%
DOWNTIME PER YEAR
5.3 minutes
PER 30-DAY MONTH
26 seconds

Each extra nine cuts the downtime by a factor of ten, and raises the cost. Choose the level the business needs. The figures are plain arithmetic, not a promise by any provider.

High availability on AWS: more than one Availability Zone



01

One instance

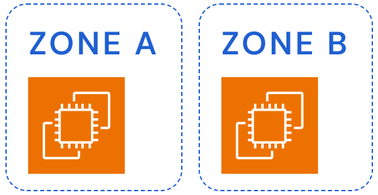
If the instance fails, the application is down.



02

Two instances, one zone

Survives an instance failure, but not a datacenter failure.



03

Instances in two zones

Survives the loss of a whole Availability Zone.

Each dashed box is an Availability Zone. They are isolated from each other, so running in at least two is the standard design.

Highly available or fault tolerant?

RECOVERS FAST

Highly available

The application recovers quickly from a failure. Users may notice a brief interruption or slowdown.

NEVER STOPS

Fault tolerant

The application keeps running with no interruption at all when a component fails. It needs more redundancy, and costs more.

Most web applications aim for high availability. Fault tolerance is for systems where any interruption is unacceptable.

AWS services that keep each layer available



DNS

Amazon Route 53

Health checks, and failover to a healthy endpoint.



TRAFFIC

Elastic Load Balancing

Spans several zones and skips unhealthy targets.



COMPUTE

Amazon EC2 Auto Scaling

Replaces failed instances and spreads them across zones.



DATABASE

Amazon RDS Multi-AZ

A standby copy in another zone takes over if the primary fails.



STORAGE

Amazon S3

Stores data across several Availability Zones.

Each layer has its own way to survive a failure. Together they remove the single points of failure. Databases come in Week 7.

Disaster recovery strategies

RTO: recovery time objective

How long the application may be down.

RPO: recovery point objective

How much recent data we can afford to lose.



Availability Zones protect against a datacenter failure. A second Region protects against a regional disaster. Each step to the right costs more.

Design principles for reliable systems

01

Recover automatically

Watch health signals and let the system repair itself, so recovery never waits for a person.

02

Practise failure

Break things on purpose in a test, so the first real failure is not the first time.

03

Scale horizontally

Many small resources do not share one point of failure, as one large resource does.

04

Stop guessing capacity

Let Auto Scaling match capacity to demand, instead of estimating it in advance.

05

Change by automation

Make changes with scripts and templates, not by hand, so they can be repeated and reversed.

These are the reliability design principles of the AWS Well-Architected Framework, in our own words.

Source: AWS Well-Architected Framework

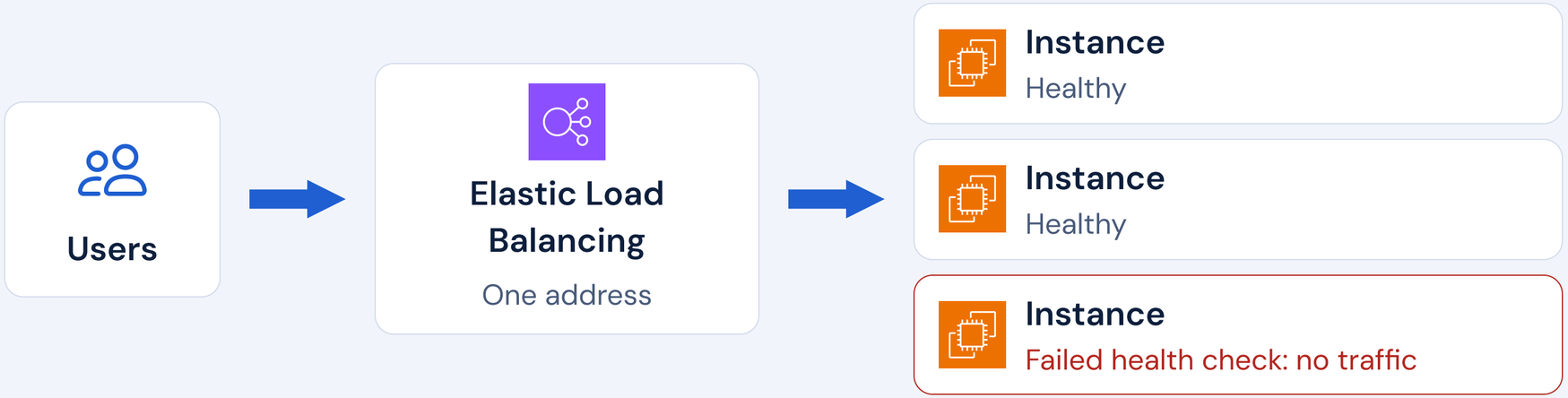
PART 3

Elastic Load Balancing

One address for users, many servers behind it.

AWS SERVICE

What a load balancer does



One entry point

Users reach one address. They never need to know how many servers exist, or which ones.

Spreads the load

Each request goes to one of the healthy targets, so no single server is overwhelmed.

Hides failures

It checks the health of every target and stops sending traffic to the ones that fail.

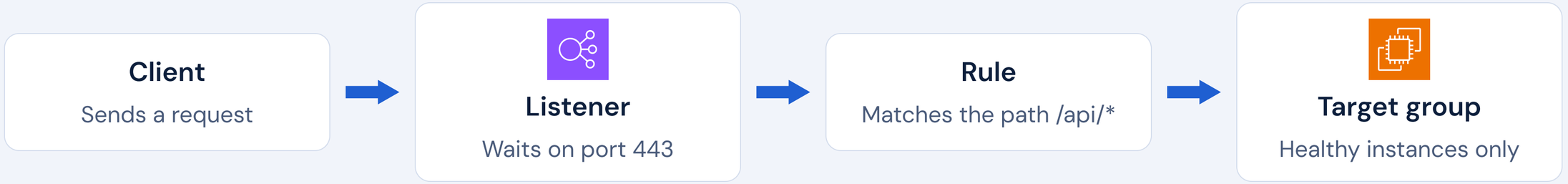
AWS SERVICE

Elastic Load Balancer types

Type	Works with	Use it for
Application Load Balancer (ALB)	HTTP and HTTPS requests (layer 7)	Websites and APIs. Can route by URL path or host name
Network Load Balancer (NLB)	TCP and UDP connections (layer 4)	Very high volumes of traffic and very low latency
Gateway Load Balancer (GWLB)	Network packets (layer 3)	Sending traffic through third-party security appliances
Classic Load Balancer	Layers 4 and 7, with fewer features	The previous generation. Not for new applications

Elastic Load Balancing is the AWS service. The layers come from the OSI model, in Week 6. For a web application, the Application Load Balancer is the usual choice.

The parts of an Application Load Balancer



01

Listener

Waits for connections on a protocol and port, for example HTTPS on port 443.

02

Rule

Decides where a request goes, for example by its path: /images or /api.

03

Target group

The set of instances that receive the requests.

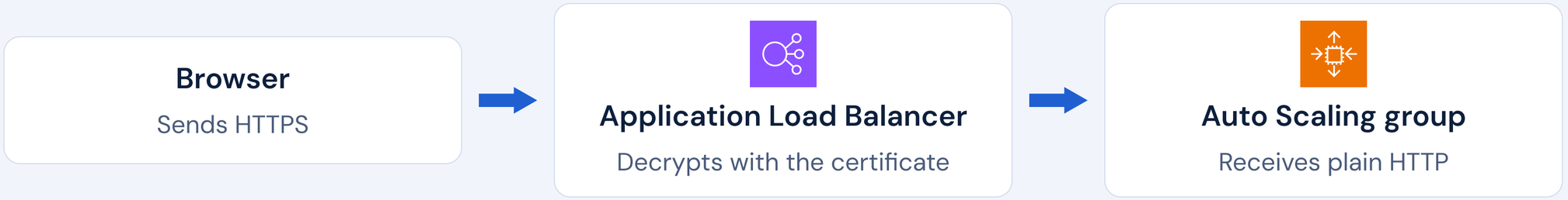
04

Health check

A request sent to each target on a schedule. Unhealthy targets get no traffic.

AWS SERVICE

HTTPS at the load balancer



AWS Certificate Manager

Issues and renews the TLS certificate for us. Public certificates for AWS services such as load balancers cost nothing.

TLS termination

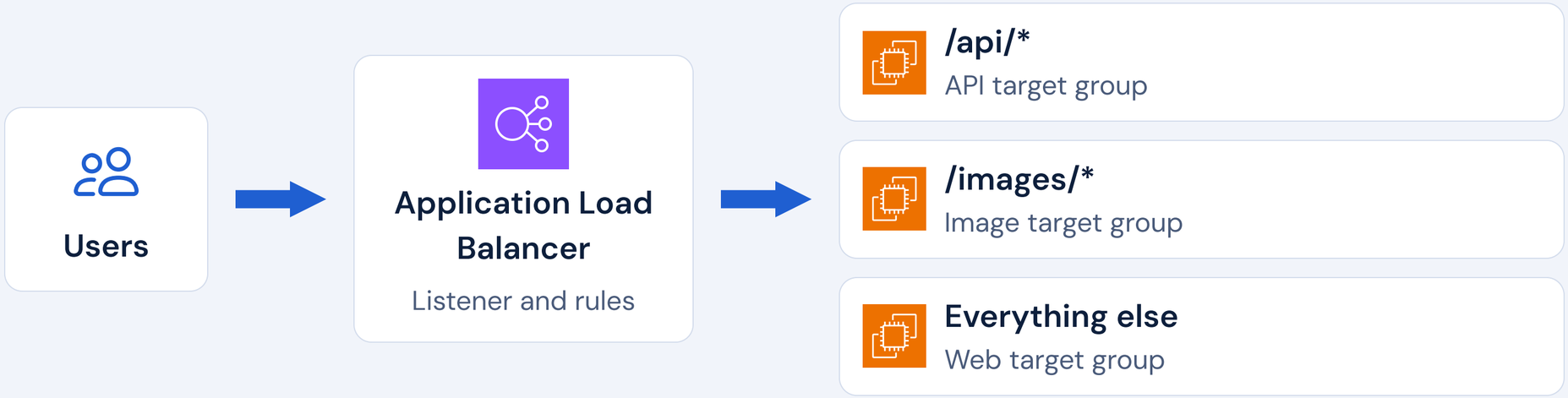
The load balancer does the encryption work once, so the instances only serve plain HTTP and stay simple.

Also works with

Amazon CloudFront and Amazon API Gateway can use the same certificates.

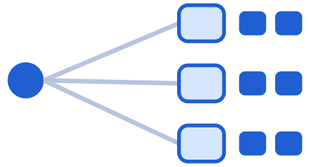
AWS SERVICE

One load balancer, several applications



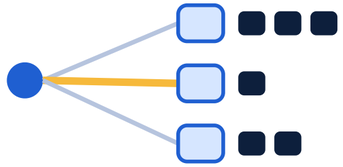
Rules send each request to the right group of servers by its path or host name. Microservices use this to share one address.

How the load balancer picks a target



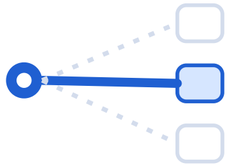
Round robin

Requests go to the targets in turn. Fair when requests cost about the same.



Least outstanding requests

The next request goes to the target with the fewest unfinished ones. Better when requests vary in length.



Sticky sessions

A cookie keeps one user on the same target. Convenient, but it works against stateless servers.



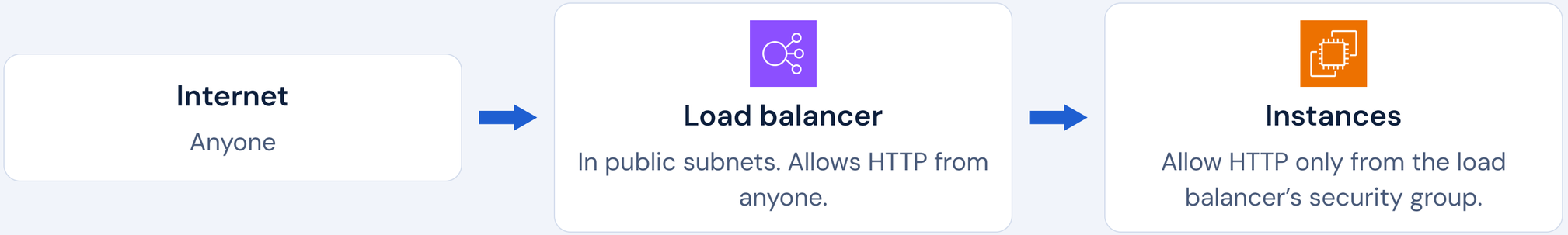
Weighted target groups

Send a share of the traffic, for example 10 percent, to a new version first.

The Application Load Balancer uses round robin by default. Least outstanding requests is a setting on the target group.

AWS SERVICE

Load balancers and security groups



In front

The load balancer sits in public subnets and accepts traffic from the internet.

Behind

The instances accept traffic only from the load balancer's security group, not from the internet.

Result

Users can reach the application, but nobody can reach an instance directly.

We look at subnets and security groups in detail in the networking session.

AWS SERVICE

Balancing between Regions: Amazon Route 53

Routing policy	Sends users to	Example
Simple	One resource	A single web server
Weighted	Resources in set proportions	90 percent to version 1, 10 percent to version 2
Latency	The Region with the lowest latency for the user	Asia-Pacific users to a Singapore Region
Failover	A standby when the primary fails its health check	A static page on S3 when the site is down
Geolocation	A resource chosen by the user's country	Content licensed for one country

DNS balances between Regions and sites. A load balancer balances between servers inside one Region.

OPEN SOURCE

Load balancers we can run ourselves

NGX

WIDELY USED

NGINX

A web server and reverse proxy that also balances HTTP traffic.

HAP

HIGH VOLUME

HAProxy

A fast load balancer for TCP and HTTP traffic.

ENV

MICROSERVICES

Envoy

A proxy built for microservices. It is the engine inside service meshes such as Istio.

TRF

CONTAINERS

Traefik

A proxy that configures itself from containers and Kubernetes.

All four are open source. We install, scale and patch them ourselves, on EC2 instances or in containers. A managed load balancer does that work for us, so most teams start there.

PART 4

Amazon EC2 Auto Scaling

The right number of instances, at every moment, without anyone watching.

AWS SERVICE

Auto Scaling groups

An Auto Scaling group keeps a set of instances at the *right number*.



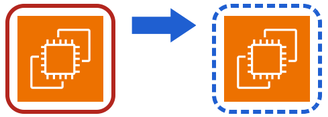
Launches

Adds instances when demand rises, from a template we define once.



Terminates

Removes instances when demand falls, so we stop paying for them.



Replaces

Detects an unhealthy instance, terminates it and launches a new one.

Three numbers define the group

2

Minimum

The group never goes below this, even when idle. Two keeps one instance in each of two zones.

2

Desired

How many instances the group aims to run right now. Scaling changes this number.

6

Maximum

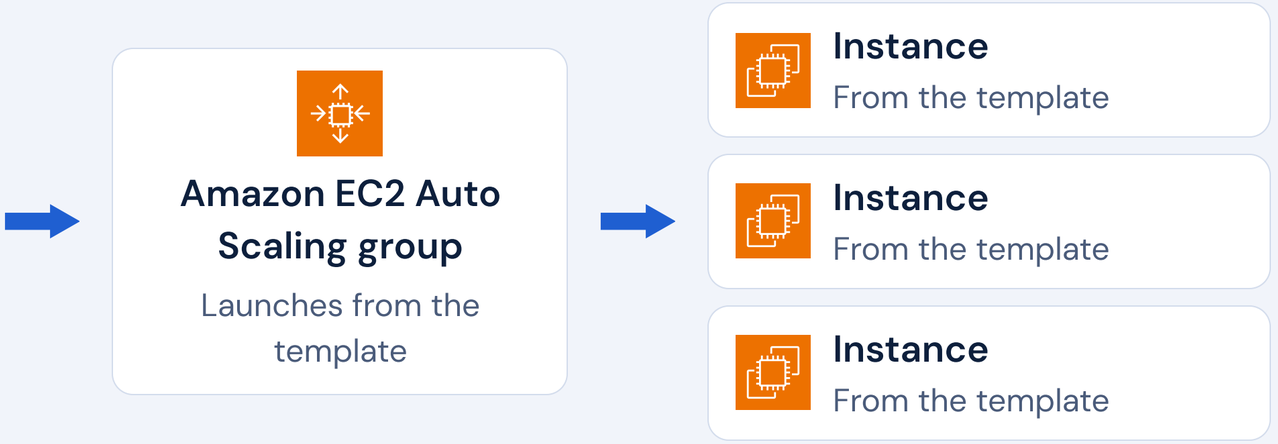
The group never goes above this, which also caps the cost.

The values are an example. The desired capacity always stays between the minimum and the maximum.

Launch templates

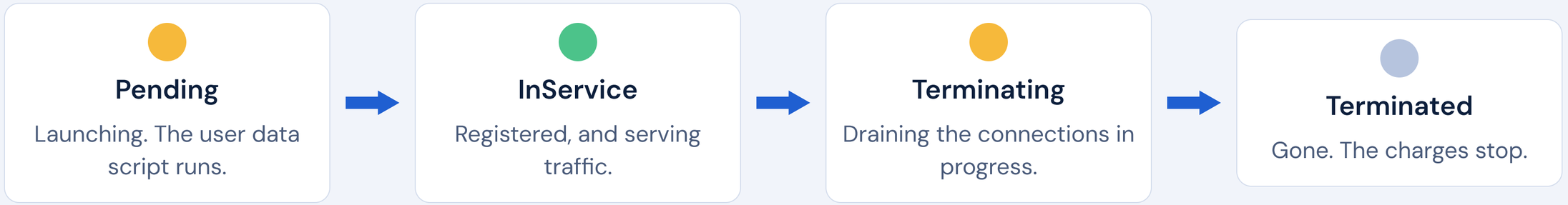
A launch template contains

- The AMI
- The instance type
- The key pair
- The security groups
- A start-up script (user data)
- On-Demand or Spot (Weeks 3 and 11)



A launch template tells the group how to launch an instance. It holds the choices we made by hand in Lab 1. Because every instance comes from the same template, every instance is identical. That is what makes them interchangeable.

The life of an instance in the group



Lifecycle hooks

Pause an instance while it starts or stops, to run our own step, such as installing software or saving logs.

Connection draining

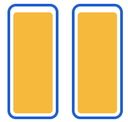
The load balancer lets requests in progress finish before the instance goes away.

Scaling policies

Policy	How it decides	Example
Manual scaling	We change the desired capacity ourselves	Raise it from 2 to 4 before a demo
Target tracking	Keeps a metric at a value we choose	Keep average CPU at 50 percent
Simple and step scaling	Adds or removes more as an alarm gets worse	Above 70 percent add one, above 90 add three
Scheduled scaling	Changes capacity at set times	Ten instances every weekday at 8:00
Predictive scaling	Forecasts load from past patterns, with machine learning	Scale out before the daily peak

Target tracking is the simplest and the usual starting point. It works like a thermostat.

Target tracking by the numbers



2

Instances now

Average CPU is 90 percent.



50%

Target

The value we asked the policy to keep.



4

Instances needed

2 × 90 ÷ 50 = 3.6, rounded up to 4.



45%

Result

The same load on 4 instances: 180 ÷ 4 = 45 percent.

The numbers are an example. The real policy is cautious about scaling in, and waits for the warm-up before its next decision.

AWS SERVICE

How the group knows when to scale



01

Measure

Amazon CloudWatch collects metrics from the instances, such as CPU utilization.



02

Alarm

When the metric crosses the threshold, a CloudWatch alarm fires.



03

Act

The scaling policy changes the desired capacity.



04

Wait

A cooldown or warm-up lets new instances start before measuring again.

Without the waiting period, the group would keep adding instances while the first new ones are still starting.

Metrics worth scaling on

Metric	What it tells us	Good for
Average CPU utilization	How busy the instances are	Compute-heavy work
Request count per target	Requests each instance receives	Web applications behind a load balancer
Network in and out	Traffic through the instances	Streaming and file transfer
Backlog per instance	Waiting jobs for each worker	Background work fed by a queue
A custom metric	Anything we publish to CloudWatch	Memory use, active sessions

Memory use is not collected by default. The CloudWatch agent publishes it as a custom metric.

Scaling on a queue



Web servers

Accept the upload.



Amazon SQS queue

Holds the jobs.



Worker instances

An Auto Scaling group.



Amazon S3

Stores the results.

Why a queue

A spike fills the queue instead of overwhelming the workers. Jobs wait a little, and none are lost.

What to scale on

The backlog per instance: waiting messages divided by running workers. Scale out when it grows.

PART 5



Scaling containers



Kubernetes, Amazon ECS and Amazon EKS: the same ideas, with containers instead of instances.

Why containers need an orchestrator

Running one container is easy. Running *hundreds* is not.



Place

Choose which machine runs each container.



Restart

Replace a container that crashes, or a machine that fails.



Scale

Add or remove containers as the load changes.



Update

Roll out a new version without taking the application down.

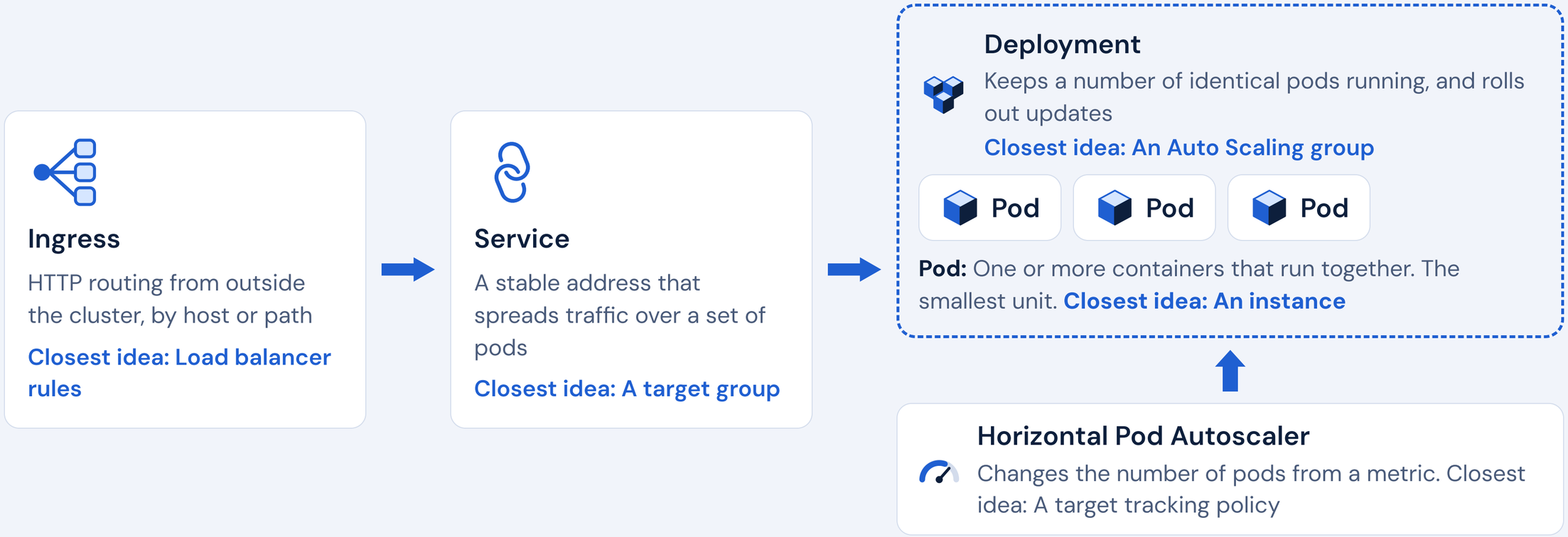
An orchestrator does these jobs from a description of what we want. Kubernetes, from Week 3, is the most widely used one.

How a Kubernetes cluster is built



We declare the desired state, for example three copies of the web container. Kubernetes keeps comparing the real state with it and fixes any difference.

Kubernetes objects, in Auto Scaling terms



A Deployment: declare what we want

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: nginx
```

replicas: 3

The desired state. Kubernetes starts three pods and keeps three running, like the desired capacity of a group.

image: nginx

The container image to run, pulled from a registry.

To scale

Change the number and apply the file again, or use the kubectl scale command.

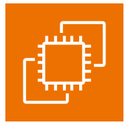
Three ways Kubernetes scales



01

Horizontal Pod Autoscaler

Adds or removes pods when a metric such as CPU crosses the target. It works like a target tracking policy.



02

Node autoscaling

Adds or removes the EC2 nodes when pods have nowhere to run, or nodes sit empty. Cluster Autoscaler and Karpenter do this.



03

Service and Ingress

Spread traffic over the pods that exist. On AWS they create Elastic Load Balancers for us.

Pods scale first, then the nodes follow: two levels of scaling, where EC2 Auto Scaling had one. Karpenter is an open source node autoscaler.

Running containers on AWS



Amazon ECR

A private registry for container images

USE IT WHEN

Always: every image needs a home



Amazon ECS

AWS's own container orchestrator

USE IT WHEN

We want the simplest way to run containers on AWS



Amazon EKS

Managed Kubernetes. AWS runs the control plane

USE IT WHEN

We want Kubernetes, or the same tools on several clouds



AWS Fargate

Runs containers with no servers to manage. Works with ECS and EKS

USE IT WHEN

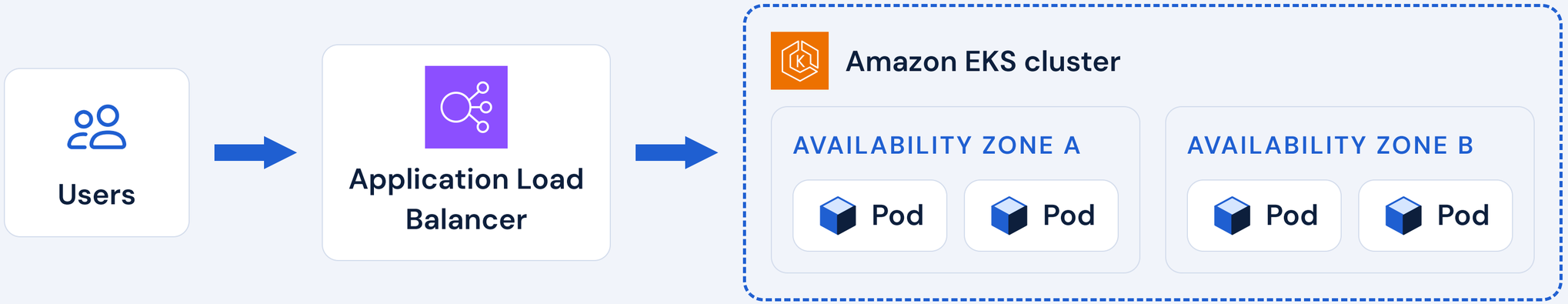
We do not want to manage nodes

Amazon ECS and EKS were the Container as a Service examples in Week 2.

Amazon ECS or Amazon EKS?

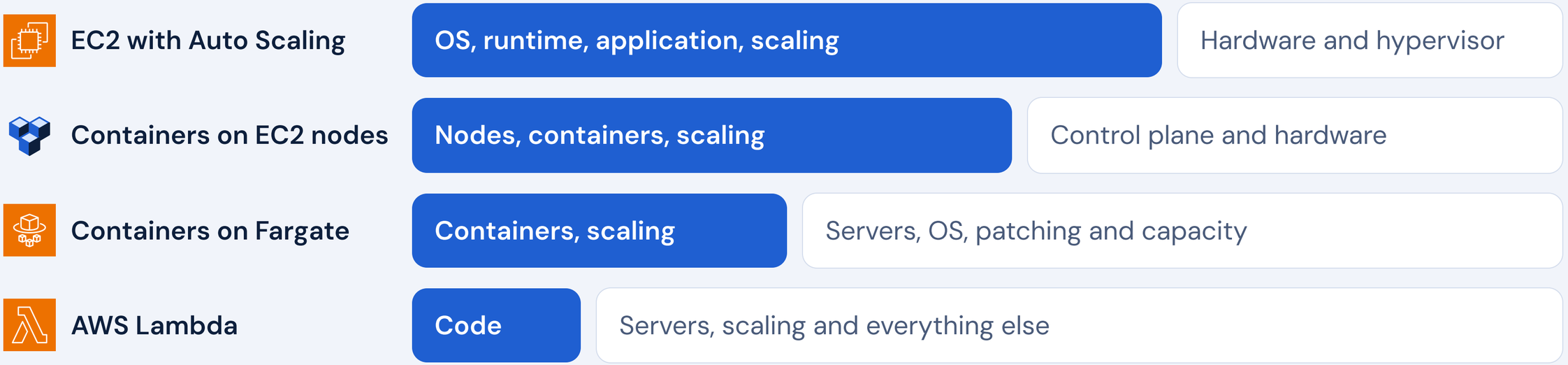
	Amazon ECS	Amazon EKS
Technology	AWS's own orchestrator	Standard Kubernetes
Learning curve	Gentler	Steeper, with more concepts
Portability	Runs on AWS only	The same files work on other clouds
Scales containers with	Service auto scaling	Horizontal Pod Autoscaler
Scales machines with	EC2 Auto Scaling, or Fargate	Node autoscaling, or Fargate

Kubernetes on AWS: Amazon EKS



An Ingress creates the Application Load Balancer and registers the pods as its targets. The Horizontal Pod Autoscaler adds pods, and node autoscaling adds EC2 nodes underneath.

The less we run, the less we manage



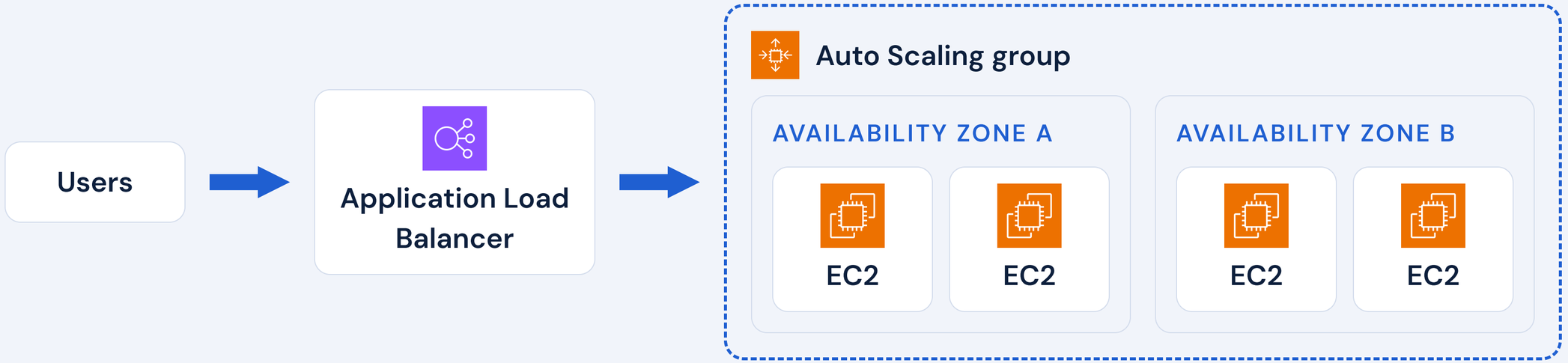
Our share in blue shrinks as we move down. The ideas stay the same: measure, decide, add or remove. Lambda returns in Week 9.

PART 6

Putting it together

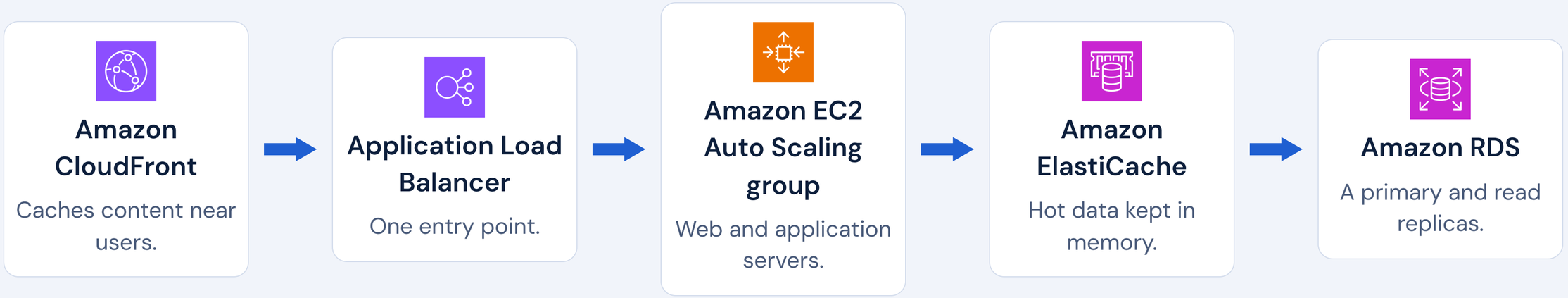
A load balancer in front, an Auto Scaling group behind, two Availability Zones underneath.

A scalable, highly available architecture



The Auto Scaling group registers each new instance with the load balancer's target group, and removes it when it terminates.

Scale every layer, not only the web servers



Cache first

A cached answer never reaches the servers or the database.

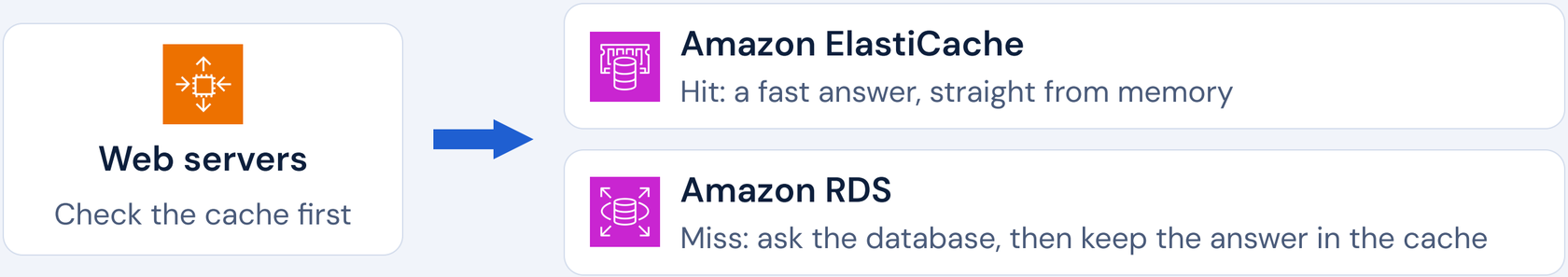
Scale out the servers

Stateless servers can be added in minutes.

Protect the database

It holds the state, so it is the hardest layer to scale. Read replicas take the reads off the primary.

A cache in front of the database



Faster

Memory answers in a fraction of the time a database query takes.

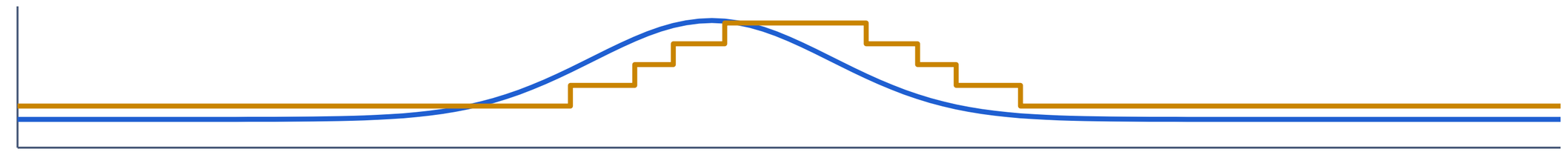
Cheaper

Every cached answer is one query the database never has to run.

Careful

A cached answer can be out of date, so give it an expiry time.

What happens during a traffic spike



01

Load rises

More users arrive. Average CPU climbs above the target.

02

Scale out

The policy raises the desired capacity. New instances launch from the template.

03

Join

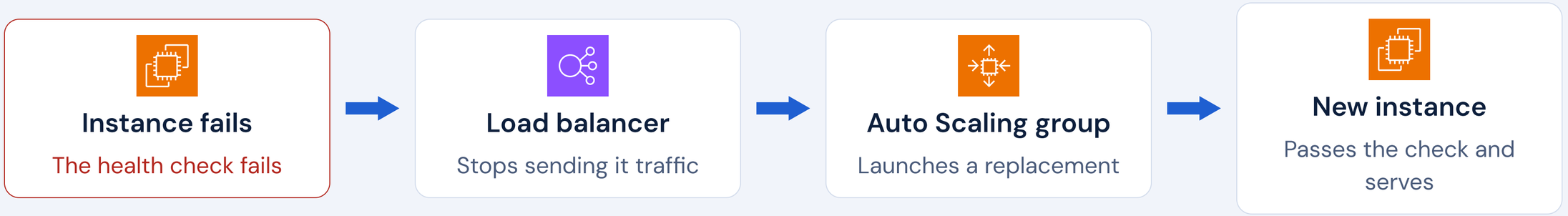
They pass the health check, and the load balancer starts sending them traffic.

04

Scale in

Load falls, the extra instances terminate, and the bill falls with them.

What happens when an instance fails



Detect

The health check fails. The load balancer stops sending requests to that instance.

Continue

The remaining instances, including those in the other Availability Zone, keep serving users.

Heal

The Auto Scaling group terminates the unhealthy instance and launches a replacement.

Nobody was paged, and nobody logged in. This is called self-healing.

Three workloads, three designs



SPIKE AT A KNOWN TIME

A results-day website

Everyone arrives at one known moment. Cache the pages in CloudFront, put the site behind a load balancer, raise the capacity with a schedule before the release, and keep target tracking for the surprise.



BACKGROUND JOBS

Video upload and encoding

Jobs arrive unevenly and can wait a little. Web servers put jobs in a queue, and a group of workers scales on the backlog. Spot Instances keep the cost low.



MANY SERVICES

A mobile app backend

Many small services grow at different speeds. Run them as containers on EKS or ECS, scale the pods with an autoscaler, and share one load balancer with path rules.

These are teaching examples, not descriptions of real systems.

Same ideas, different names

Idea	AWS	Microsoft Azure	Google Cloud
Auto scaling of servers	EC2 Auto Scaling	Virtual Machine Scale Sets	Managed instance groups
Load balancer	Elastic Load Balancing	Load Balancer, Application Gateway	Cloud Load Balancing
Managed Kubernetes	Amazon EKS	Azure Kubernetes Service	Google Kubernetes Engine
Containers with no servers	AWS Fargate	Azure Container Apps	Cloud Run
Metrics and alarms	Amazon CloudWatch	Azure Monitor	Cloud Monitoring

Learn the idea once, then look up the name. Kubernetes itself is the same on all three.

What it costs



Amazon EC2 Auto Scaling

No additional charge for the service itself. We pay for the instances it launches.



Elastic Load Balancing

Charged for each hour the load balancer runs, plus the traffic it handles, even with no instances behind it.

Set a sensible maximum on the group, and delete the load balancer when a lab is finished.

Lab 2: Load balancer and Auto Scaling group

1. Create security groups for the load balancer and the instances.
2. Create a target group and an Application Load Balancer.
3. Create a launch template and an Auto Scaling group attached to the load balancer.
4. Test the load balancer, and watch a failed instance get replaced.
5. Clean up everything, in reverse order.

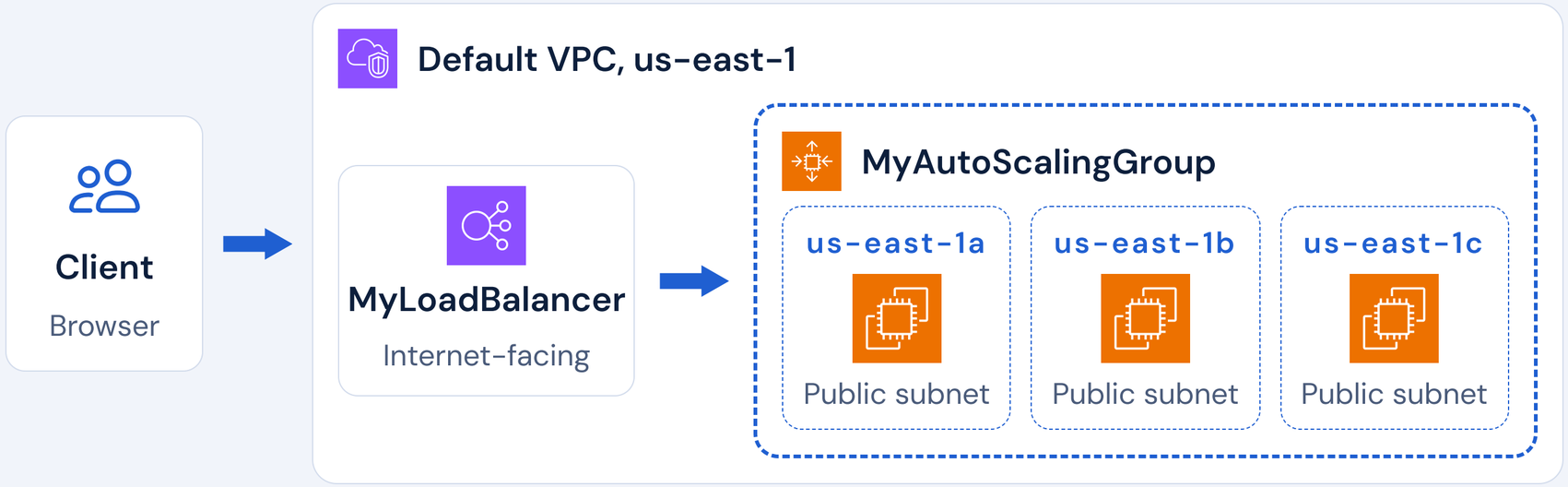
REMEMBER

What to submit

Individual work in your AWS Academy Learner Lab, in us-east-1. A report README.md and 14 screenshots in cc-lab-YOUR_ID/lab2/ of your GitHub repository. Every console screenshot shows your Learner Lab user name. Watch your usage against the 50 USD quota.

Source: Lab 2 document

What you will build



A client reaches one load balancer. It spreads the requests over the instances that the Auto Scaling group keeps running in three Availability Zones.

Source: Lab 2 document

The tasks and the names to use

	Task	What you create	Key settings
	1. Security groups	LoadBalancerSecurityGroup, LaunchTemplateSecurityGroup	HTTP from anywhere. SSH too on the second
	2. Target group	LoadBalancerTargetGroup	HTTP, port 80, default health checks
	3. Load balancer	MyLoadBalancer	Internet-facing, all zones, HTTP port 80
	4. Launch template	AutoScalingLaunchTemplate	Amazon Linux 2023, t2.micro, user data
	4. Auto Scaling group	MyAutoScalingGroup	Attach the target group. Desired 2, minimum 2, maximum 3

Use these names exactly. Other names may be refused in the Learner Lab. Task 5 tests it, and Task 6 cleans up.

Source: Lab 2 document

The launch template's user data

Paste this script under Advanced details, User data, when you create the launch template.

```
#!/bin/bash
# Enabling admin rights
sudo su
# Updating system and installing httpd
yum update -y
yum install -y httpd.x86_64
# Launching a simple httpd server
systemctl start httpd.service
systemctl enable httpd.service
# Writing HTML file to serve on the httpd server
echo "Hello World from ($(hostname))" > /var/www/html/index.html
```

Why a host name

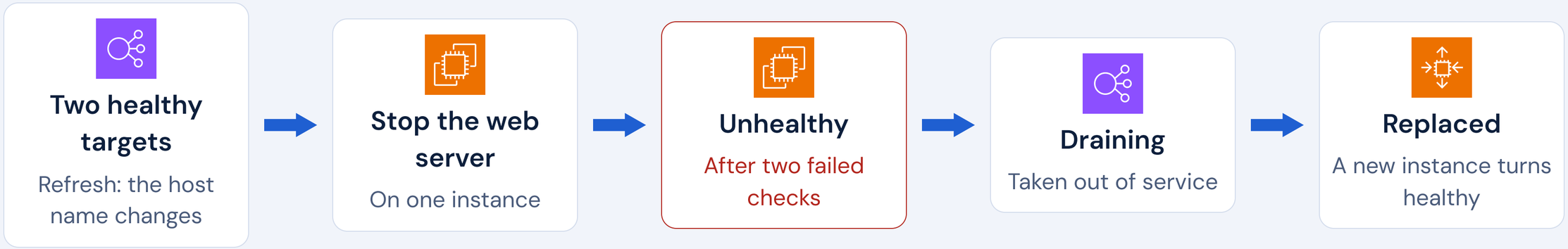
Every instance writes its own name on the page. Refresh the load balancer's DNS name, and the name changes as requests reach different instances.

A lab shortcut

The lab opens HTTP and SSH to anywhere to keep it simple. In production, instances accept HTTP only from the load balancer's security group.

Source: Lab 2 document

Task 5: watch a failed instance get replaced



Screenshots 12 to 14 show this sequence. An Application Load Balancer checks the application, not only the instance, so a stopped web server is enough to fail the check.

```
sudo systemctl stop httpd.service
```

Source: Lab 2 document

Optional extension: make the group scale out

1. Open MyAutoScalingGroup, then Automatic scaling, and create a target tracking policy on average CPU utilization with a target of 40.
2. Connect to one instance with EC2 Instance Connect and start one busy process. A t2.micro has one vCPU.
3. Watch the Activity tab. After a few minutes the desired capacity rises from 2 to 3, the maximum.
4. Stop the load, and watch the group scale back in later.

```
yes > /dev/null &
```

To stop the load: pkill yes

REMEMBER

Optional

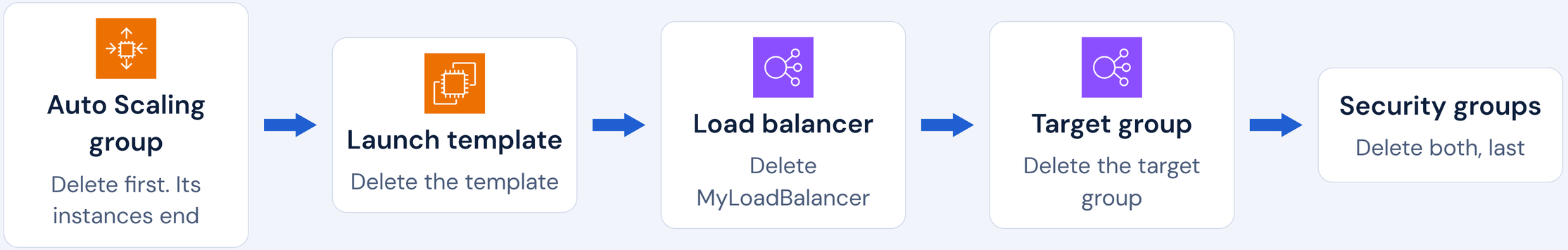
Not part of the 14 screenshots. The policy is deleted together with the group at clean-up.

When the lab does not work

Symptom	Likely cause	What to check
Targets stay initial or unhealthy	The instances are still starting, or the instance security group has no HTTP rule	Wait a few minutes. Check the inbound rules
The browser times out	The load balancer’s security group has no HTTP rule, or it is not Active yet	Check the rule and the load balancer state
A stopped server is never replaced	Elastic Load Balancing health checks are off in the group	Edit the group, Health checks, turn them on
Resources will not delete	They were deleted out of order	Group first, then template, load balancer, target group, security groups

Source: Lab 2 document

Clean up in reverse order



Take all 14 screenshots first. Deleting the group terminates its instances, which can take up to ten minutes. A load balancer left running keeps costing money.

Source: Lab 2 document

SCENARIO

Exam results go online at 9:00, and the website stops working.

Thousands of students open it at the same moment, and it runs on one small server. What would you change to keep it working? Use the tools from today, and say which comes first.

One possible answer

1. Add a second server in another Availability Zone, and put an Application Load Balancer in front of both.
2. Let an Auto Scaling group add servers by itself: more before 9:00 on a schedule, and more whenever the CPU gets busy.
3. Cache the pages that do not change with Amazon CloudFront, so fewer requests reach the servers.
4. Keep each student’s session outside the servers, so any server can answer any request.

THINK IN LAYERS

Why this order

First remove the single point of failure. Then let the capacity follow the demand. Then reduce the load itself.

Tool cheat sheet

We need to	On AWS	Open source and other clouds
Add and remove servers	EC2 Auto Scaling	Scale Sets (Azure), instance groups (Google)
Spread web traffic	Application Load Balancer	NGINX, HAProxy, Envoy, Traefik
Run containers	Amazon ECS, EKS, Fargate	Kubernetes, AKS, GKE
Scale containers	Service auto scaling, HPA	Kubernetes HPA, Cluster Autoscaler, Karpenter
Watch the metrics	Amazon CloudWatch	Prometheus and Grafana

GROUP ACTIVITY

Design for opening day

Teams of four. About 25 minutes of work, then a few minutes per team to share.

The scenario: tickets for the Campus Show

IMAGINE

A fictional scenario

The student union sells tickets for its yearly show on one website. Sales open at 12:00 on a fixed day. Last year the site ran on one small server and stopped working within minutes. This year the union wants it to stay up.

The constraints

- Almost everyone arrives at noon. Traffic drops within the hour.
- Every order must be saved. None may be lost.
- The site must stay up if one Availability Zone fails.
- A small budget, and two volunteers with no night shift.
- Use what we met today, with Amazon S3 and a database.

What each team does

1 · Sketch
10 minutes: boxes and arrows

2 · Scale
6 minutes: policy and numbers

3 · Break it
5 minutes: what fails next?

4 · Share
2 minutes per team

Architect

Draws the traffic path, from the user to the data.

Scaling lead

Chooses the policy, the metric, and the minimum, desired and maximum.

Reliability checker

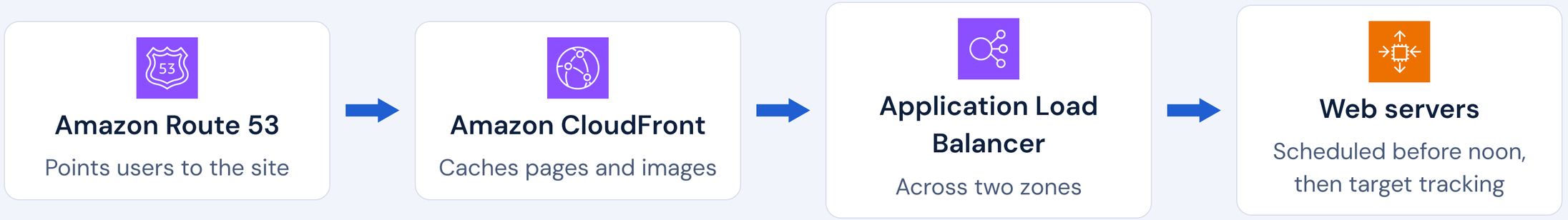
Keeps asking what happens when a zone, an instance or the database fails.

Presenter and timekeeper

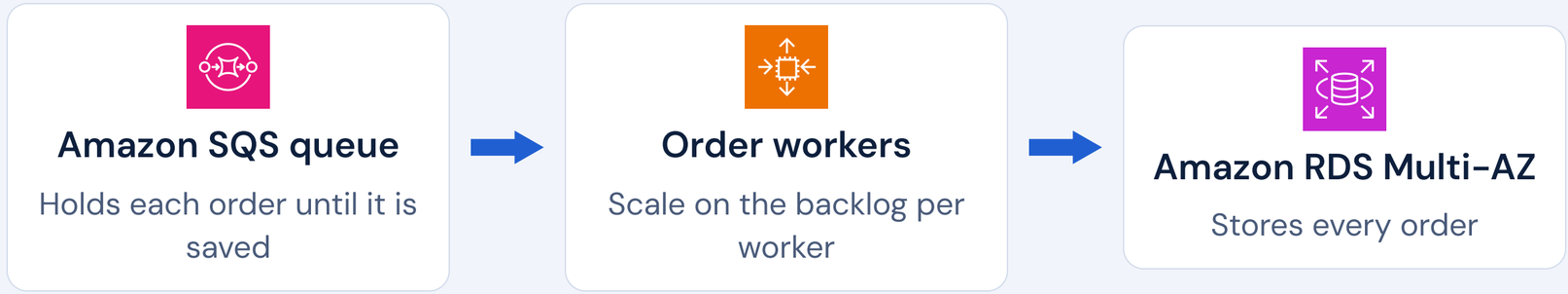
Explains the design in two minutes, and watches the clock.

Hand in one sketch with the traffic path, the scaling policy with its numbers, and the answer to: what if one zone fails?

One possible design



Each order then goes into a queue, so the noon burst never reaches the database directly:



Check the design

Ask the team	Look for
Is there a single point of failure?	Two or more of everything, in two zones
How does it scale?	A metric that matches the work, with a minimum and a maximum
Where does the state live?	Orders in a database and files in S3, not on the instances
What happens at noon?	A cache or queue that absorbs the burst, and capacity raised before 12:00
What does it cost?	A maximum on the group, and scaling in after the sale

Compare with the reference design. A different design is fine if the reasons hold.

Key takeaways

01

Scale out, not just up

Horizontal scaling has no practical limit and survives failures, if the application is stateless.

02

Two zones, always

High availability means no single point of failure, including the datacenter.

03

Load balancer in front

One entry point that spreads traffic and hides unhealthy instances.

04

Auto Scaling behind

The right number of instances at every moment, replaced automatically when they fail. Containers scale the same way, with Kubernetes, ECS or EKS.

QUIZ

Post-knowledge quiz

Scaling, high availability, load balancing, Auto Scaling groups and container orchestration.

Week 4

Next session



01

Cloud storage



02

Lab 3: storage